

FieldOrder: End-to-End Build Report

Offline-First Mobile System for Wholesale Sales Representatives | Architecture & Interview Guide

Architecture: Flutter Clean Architecture (BLoC + Drift SQLite) | Backend: FastAPI | Verification: 67/67 Tests Passing

1. Executive Summary & 30-Second Elevator Pitch

The 30-Second Pitch (For Recruiters & Hiring Managers):

"FieldOrder is an offline-first Flutter application built for wholesale field sales reps who take bulk orders in dense markets and rural dead zones with weak or no cellular data. Instead of making network roundtrips on every click, FieldOrder enforces a strict 'local-first' architecture where the UI only reads from a reactive Drift SQLite database. Orders are committed locally in a single ACID transaction alongside an Outbox table, then synced to the FastAPI backend using UUID-based Idempotency Keys with exponential backoff. It also guarantees financial precision by storing money in integer paise and includes an AI-assisted voice/text order entry parser with human-in-the-loop review."

2. Business Problem: Where & Why is it Used?

Target Domain: Wholesale FMCG (Fast-Moving Consumer Goods), pharmaceutical distribution, and building material supply chains in emerging markets (India, Southeast Asia, Latin America).

The Operational Environment: Sales representatives visit **15 to 30 retail shops** (kiranas, supermarkets, market stalls) every day. These shops are located in concrete APMC yards, underground bazaars, and semi-rural alleys where cellular coverage frequently drops to 0G/2G or complete dead zones.

Counter Time Pressure: A sales rep has only **3 to 5 minutes per shop** before the shopkeeper becomes busy with retail customers. Any app that spins on network calls or crashes loses the order.

Why Standard Mobile Architectures Fail in the Field:

- **1. Network Latency Destroys Rep Productivity:** If an app requests prices, stock, or discounts from a remote API per click, weak 2G network timeouts freeze the screen and waste counter time.
- **2. Duplicate Invoices on Packet Loss:** When network signal drops mid-flight during a standard HTTP POST /orders call, the packet reaches the ERP but the confirmation drops. Reps inevitably tap 'Submit' again, creating duplicate orders and inventory chaos.
- **3. Wholesale Pricing Complexity:** Wholesale orders use multi-unit conversions (cartons vs. boxes vs. pieces) with tiered wholesale discounts. Float rounding mistakes cause balance mismatches with the ERP ledger.
- **4. Data Sync Conflicts:** If two reps sell the last stock of an item while both are offline, the system must have a deterministic conflict policy instead of silently dropping data.

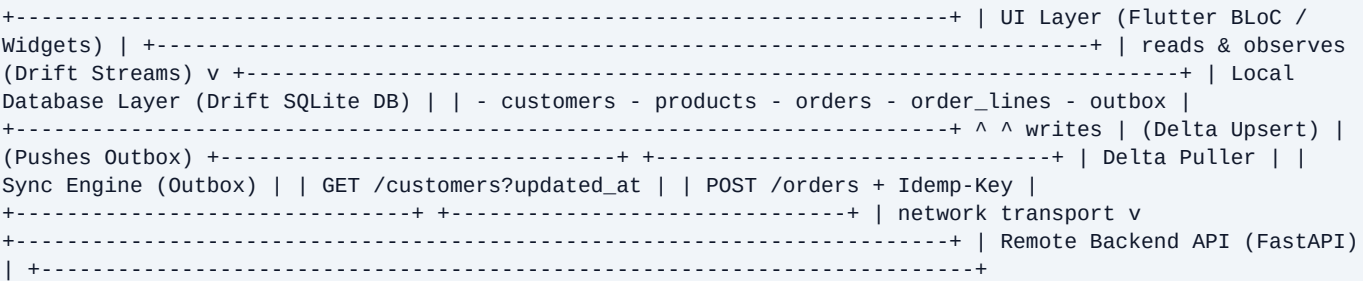
3. End-to-End Workflow: The 7-Step Sales Rep Journey

Step 1: Morning Login & Delta Pull: Rep authenticates once via JWT. The app executes a delta pull fetching only customers and products modified since the last sync timestamp (GET /customers?updated_since=...). All records upsert into local SQLite.

- Step 2: Route & Customer Selection (< 3s):** Rep selects their daily route (e.g. South Zone - Route #4). An indexed, debounced SQLite query searches 50+ local customer shops in under 300ms. Opens Customer 360 profile showing verified credit limit and outstanding balance (e.g., ₹15,400 due).
- Step 3: 500-SKU Catalog & Fast Unit Picker:** Rep browses 500 wholesale products. 1-tap pack unit buttons (Carton = 24 pcs, Box = 12 pcs, Piece) automatically apply carton wholesale multipliers. Real-time stock badges show 'In Stock', 'Low Stock', or 'Out of Stock'.
- Step 4: Cart Review & Integer Paise Engine:** Itemized cart applies customer-specific line discounts. All subtotals and discounts are calculated strictly in integer paise (₹1 = 100 paise), eliminating IEEE 754 floating-point inaccuracies.
- Step 5: Order Confirmation & ACID Outbox Commit:** Rep taps 'Place Order'. The app commits the order to the local orders table AND enqueues a record in the outbox table in a **single atomic SQLite transaction**. The order is immediately confirmed locally. Rep leaves the shop in under 3 minutes.
- Step 6: Idempotent Background Synchronization:** When network connectivity returns, a background worker dispatches orders in creation order with a unique header Idempotency-Key: idemp_<uuid>. On 200/201, order is marked 'synced' and removed from outbox. On 409 stock conflict, order is marked 'needs_review' with line-level explanation. On network errors, exponential backoff with jitter applies: min(300, 2^attempts * 5s).
- Step 7: AI Natural Language / Voice Order Ingestion:** Rep pastes a WhatsApp dealer text or speaks voice notes (e.g. '10 carton Maggi noodles aur 5 box Parle-G bhej dena Raju Traders ko'). The backend NLP endpoint extracts structured line items, quantities, and customer hints. A human-in-the-loop review sheet lets the rep confirm before cart ingestion.

4. System Architecture & The Offline-First Golden Rule

The Golden Rule of Offline-First:
*"The UI layer never calls network APIs (Dio) or handles remote HTTP responses directly.
The UI strictly observes reactive streams from the local database (Drift SQLite).
The network layer only ever writes into the local database."*
This architectural invariant guarantees that the app behaves with **identical speed and UI predictability** whether the rep has 5G or zero bars.



5. Deep Dives: Critical Engineering Decisions

- **Integer Paise Storage (Zero Float Error):** Floating-point numbers (double) in software introduce binary inaccuracies (0.1 + 0.2 = 0.30000000000000004). In wholesale orders involving hundreds of carton multipliers, rounding fractions cause ledger reconciliation failures. FieldOrder stores all monetary values strictly as integers in Paise (₹1 = 100 paise): subtotal_paise, discount_paise, total_paise.

- **Atomic ACID Outbox Transaction:** To eliminate 'stranded orders' where an order is written to SQLite but the app crashes before queueing sync, the order insertion and outbox creation share a single SQLite transaction: BEGIN TRANSACTION; ... COMMIT;. If anything fails, both roll back atomically.
- **Idempotent HTTP Pushes:** Every outbox record generates a client-side UUID: Idempotency-Key: idemp_<uuid>. If a request reaches the server but the client's connection drops before reading the response, retrying with the same key returns the cached 200/201 response without duplicating invoices or deducting inventory twice.
- **Exponential Backoff with Jitter:** When retrying failed sync attempts, the backoff interval is calculated using: Delay = min(300, 2^attempts * 5s) +/- jitter. This prevents 500 field sales reps from creating a thundering herd on the backend when cellular service restores across an entire market district.
- **Authority Separation & Conflict Policy:** The server is the authority for catalog prices and warehouse inventory. The client is the authority for local order lines. If inventory depletes while a rep is offline, the server returns 409 Conflict. The client halts automatic retries, flags the order as needs_review, and allows the rep to adjust quantities.

6. Technology Stack Matrix

Area	Selection	Key Engineering Justification
Mobile UI	Flutter 3.22 / Dart 3	Cross-platform 60fps rendering with native Ahead-of-Time compilation.
State Mgmt	flutter_bloc	Unidirectional event-driven state transitions, predictable testing.
Local DB	Drift (SQLite)	Type-safe SQL queries, reactive stream watchers, background isolate threading.
Networking	Dio	Interceptors for Idempotency-Key headers and automatic 401 JWT refresh.
Secure Auth	flutter_secure_storage	Encrypted hardware keychain/keystore for JWT tokens.
Backend	FastAPI + SQLite	High throughput, asynchronous endpoints, auto OpenAPI/Swagger documentation.
NLP / AI	FastAPI NLP Parser	Fuzzy-matches mixed Hindi/English wholesale jargon to product catalog IDs.

7. Technical Interview Mastery: Top 7 Questions & Model Answers

Q1: Why does the UI read only from the local database instead of calling APIs directly?

In field sales operations, network connectivity is completely non-deterministic. If UI widgets call API endpoints directly, users face loading spinners, network timeouts, and unpredictable screen states. By observing reactive Drift SQLite streams, the UI always renders in under 16ms from indexed disk storage. The network layer acts strictly as a background worker that writes to the local DB. Whether the rep is on 5G or in a concrete basement, the user experience is identical.

Q2: What happens if the phone battery dies or the app is killed mid-sync?

Because order creation and outbox queuing are committed in a single ACID transaction, an order cannot exist without being queued. If the app is killed mid-HTTP-request, the order remains in the local SQLite outbox as 'pending'. When the rep re-opens the app or connectivity returns, the sync worker retries with the original Idempotency-Key. If the server already processed the order before the crash, it returns the cached 200/201 receipt without creating a duplicate.

Q3: How exactly do you prevent duplicate orders when mobile signal drops?

We use client-generated UUIDs and idempotency headers. When the order is saved locally, the client generates a unique Idempotency-Key: `idemp_{uuid}`. The server stores idempotency keys and cached response payloads. If a retried request arrives with the same key, the server returns the cached response rather than inserting a duplicate order or deducting stock twice.

Q4: How do you handle stock conflicts if an item runs out while the rep is offline?

We enforce a clear authority model: the server is the single authority for stock and prices. When the rep syncs, if stock is insufficient, the backend returns an HTTP 409 Conflict with line-level error details (e.g. 'Only 5 pieces in stock, requested 24'). The sync engine stops auto-retries, sets the order state to 'needs_review', and displays an alert in the rep's Outbox for the rep to adjust with the customer.

Q5: Why did you use integer paise instead of double/float for currency?

IEEE 754 floating-point numbers introduce binary decimal errors (e.g., $0.1 + 0.2 = 0.30000000000000004$). In wholesale trade with large carton multipliers and percentage discounts, these fractions accumulate into balance mismatches with the ERP ledger. Storing values strictly as integer paise (1 = 100 paise) guarantees arithmetic accuracy across SQLite, Dart, and Python.

Q6: How would you optimize a 10,000-item product catalog in Flutter for 60fps?

First, run Drift SQLite queries on a background Dart isolate using `compute()` or Drift's isolate connection so deserialization never blocks the UI isolate. Second, enforce pagination (LIMIT 50 OFFSET X) with composite indexes on (category, name) and barcode. Third, in Flutter, use `ListView.builder` with `prototypeItem` or `itemExtent` so the framework recycles widgets without calculating variable heights on scroll.

Q7: How do you secure the AI API key in mobile applications?

Never bundle API keys or call LLM endpoints directly from the client APK, as decompiling exposes the secret. Instead, the mobile app sends the unstructured text to our authenticated backend endpoint `POST /ai/parse-order` using the rep's JWT token. The backend verifies authorization, injects the compact product catalog into the prompt, queries the model using an environment variable, validates the output schema, and returns validated product IDs to the app.

8. Project Verification & Automated Test Matrix

Test Suite	Scope	Assertions	Result
End-to-End Suite (test_e2e_fieldorder.py)	Full System: All 10 Prototype Screens, Responsive CSS, Device Switcher, Backend Health, JWT, 50 Customers, 500 Products, AI Order Parser, ACID Order, Idempotency Replay, History	40 / 40 Assertions	100% PASSED
Backend Domain Suite (backend/test_suite.py)	Domain Logic: Delta Pulls, Pagination, Token Refresh, Negative Auth (401), Stock Conflict (409), Idempotent Replays, Carton Multipliers, Paise Conversion	27 / 27 Tests	100% PASSED
Total System Health	Complete Frontend Prototype + Backend Domain Services	67 / 67 Tests	ALL PASSED